

Docs

SpikeGPT: A Spiking Neural Language Model for Chatbot Systems

References:

<https://arxiv.org/pdf/2302.13939>

Code:

<https://github.com/ridgerchu/SpikeGPT>

Slide:

https://www.canva.com/design/DAG1HE-CxNs/cQjn7yyAoxWf6Gd2Vwe4Rg/edit?utm_content=DAG1HE-CxNs&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Slide Đồ án:

<https://canva.link/9mv9m4g0nkiief2>

Nội dung thuyết trình

Nội dung

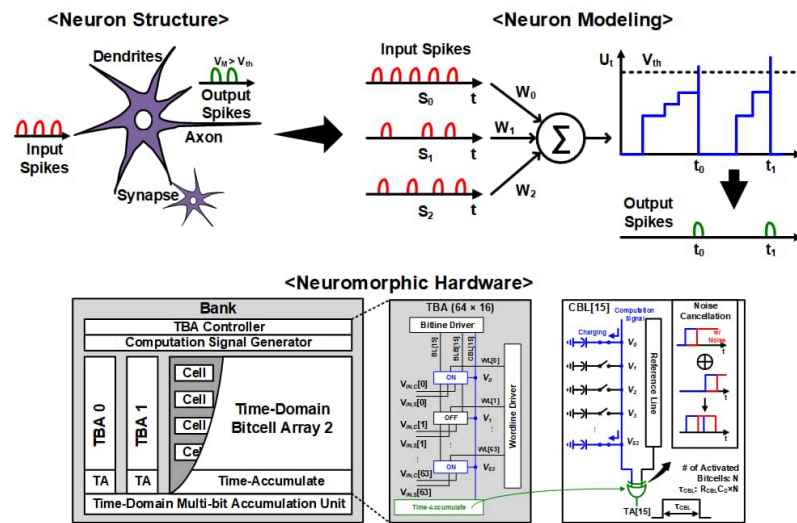
1. Giới thiệu về SpikeGPT
2. Các vấn đề dẫn đến sự ra đời của SpikeGPT
3. Kiến trúc SpikeGPT
 - 3.1 Leaky Integrate-and-Fire Neuron
 - 3.2 Model Architecture
 - 3.3 Binary Embedding
 - 3.4 Efficient Processing of Variable-Length Sequences Using Spiking RWKV
 - 3.5 Spiking Receptance Feed-Forward Networks (SRFFN)
 - 3.6 Theoretical Energy Consumption Analysis

-
4. *Experiments*
 - 4.1 *Results on Natural Language Generating Tasks*
 - 4.2 *Results on Natural Language Understanding Tasks*
 - 4.3 *A Study of SpikeGPT and RWKV Variants*

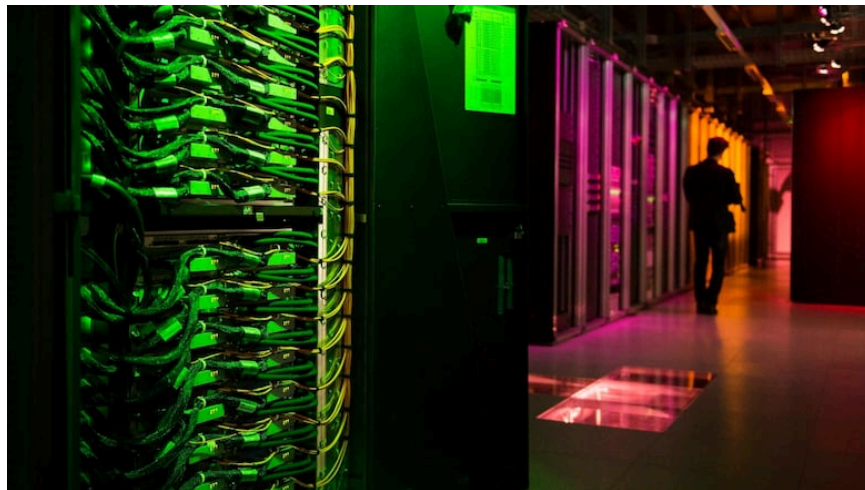
-
5. SpikeGPT for Chatbot in ecommerce
 - 5.1. Giới thiệu
 - 5.2. Dataset
 - 5.3. Preprocessing
 - 5.4. Training & Evaluation
 - 5.5. Demo
 6. Conclusion

Hy

1. Giới thiệu về SpikeGPT



Trong bối cảnh công nghệ hiện đại, sự gia tăng quy mô theo cấp số nhân của các mô hình ngôn ngữ lớn đã phát sinh những hệ lụy đáng kể đối với bài toán tiêu thụ năng lượng. Tiến trình huấn luyện và duy trì vận hành các mô hình này tại những trung tâm dữ liệu quy mô lớn đòi hỏi một nguồn điện năng khổng lồ. Thực trạng này thiết lập một giới hạn kỹ thuật đối với việc triển khai các hệ thống trí tuệ nhân tạo tiên tiến trên các thiết bị biên – môi trường vốn dĩ tồn tại những ràng buộc nghiêm ngặt về công suất, dung lượng pin và giới hạn nhiệt độ (điển hình như thiết bị di động, hệ thống robot tự hành hoặc các mạng lưới cảm biến IoT).



Khác biệt với nguyên lý vận hành của các Mạng nơ-ron nhân tạo ANN truyền thống – vốn xử lý các ma trận số thực liên tục với mật độ tính toán dày đặc – SNN hoạt động dựa trên cơ chế truyền tải các tín hiệu nhị phân rời rạc (chỉ bao gồm hai trạng thái 0 và 1), hay còn gọi là các **xung**.

Tính đến thời điểm hiện tại, các biến thể kiến trúc của SpikeGPT đã được phát triển và kiểm chứng thực nghiệm thành công ở các thang mô hình 46 triệu, 216 triệu và 260 triệu tham số. Mặc dù cấu hình này vẫn duy trì ở mức độ tương đối khiêm tốn khi đối chiếu với các LLM quy mô hàng trăm tỷ tham số, đây vẫn được giới chuyên môn đánh giá là một thành tựu mang tính nền tảng đối với tiến trình phát triển của SNN. Các kết quả nghiên cứu đã cung cấp những bằng chứng thực nghiệm quan trọng, góp phần giảm bớt quan điểm khoa học truyền thống cho rằng SNN chỉ khả thi đối với các tác vụ thị giác máy tính, đồng thời xác lập năng lực xử lý ngữ nghĩa phức tạp của mạng SNN với mức chi phí tính toán được tối thiểu hóa.

Hệ thống SpikeGPT được phát triển nhằm cung cấp một giải pháp nền tảng cho thách thức nêu trên. Xét trên phương diện cấu trúc kỹ thuật, SpikeGPT được ghi nhận là **mô hình ngôn ngữ tạo sinh (Generative Language Model) đầu tiên vận hành hoàn toàn trên nền tảng Mạng nơ-ron xung SNNs**, đồng thời cung cấp minh chứng thực nghiệm về khả năng ứng dụng thuật toán lan truyền ngược trên các tập ngữ liệu văn bản quy mô lớn.

2. Sự Ra Đời của SpikeGPT

2.1. Thách thức năng lượng của LLM truyền thống và Tiềm năng của SNN

Các mô hình Ngôn ngữ Lớn (LLM) hiện tại, điển hình là GPT, đang đối mặt với vấn đề **tiêu thụ năng lượng và chi phí tính toán cực kỳ lớn** do quy mô ngày càng tăng (hàng tỷ tham số). Điều này tạo ra rào cản lớn cho việc triển khai chúng trên các thiết bị giới hạn về tài nguyên như thiết bị biên (edge devices), robot hay điện thoại thông minh.

Trong bối cảnh đó, **Mạng thần kinh Spike (Spiking Neural Networks - SNN)** nổi lên như một giải pháp hứa hẹn:

- **Nguyên tắc hoạt động:** SNN mô phỏng hoạt động của bộ não sinh học.
- **Hiệu quả năng lượng:** SNN **tiết kiệm năng lượng hơn** Mạng thần kinh Nhân tạo (ANN) truyền thống nhờ cơ chế:
 - Tính toán **hướng sự kiện (event-driven)** và **thưa thớt (sparse)**.
 - Neuron chỉ **kích hoạt ("bắn spike")** khi cần thiết.
- **Phù hợp:** SNN lý tưởng cho **phần cứng thần kinh mô phỏng (neuromorphic hardware)** và các ứng dụng **AI biên (edge AI)**.

2.2. Khoảng trống ứng dụng và Khó khăn trong việc chuyển đổi

Mặc dù SNN đã được áp dụng thành công trong nhiều lĩnh vực như nhận dạng hình ảnh (MNIST), phát hiện đối tượng và robot học, vẫn tồn tại một **khoảng trống lớn: hầu như chưa có ứng dụng SNN nào cho tác vụ sinh ngôn ngữ (language generation)**.

Việc chuyển đổi một mô hình ngôn ngữ truyền thống sang kiến trúc SNN gặp phải thách thức lớn:

- **Mô hình Transformer** sử dụng cơ chế **self-attention**, đòi hỏi **phép nhân ma trận lớn** và mật độ tính toán cao.
- **SNN** lại hoạt động dựa trên **các spike temporal (theo thời gian)**.
- **Kết quả:** Kiến trúc Transformer **không thể tích hợp trực tiếp** với SNN một cách hiệu quả.

2.3. SpikeGPT - Giải pháp dựa trên Kiến trúc RWKV

Để khắc phục các vấn đề trên và hiện thực hóa mục tiêu sử dụng SNN cho tác vụ sinh ngôn ngữ, dự án **SpikeGPT** ra đời với các mục tiêu:

- **Duy trì** khả năng sinh ngôn ngữ mạnh mẽ của LLM.
- **Giảm thiểu đáng kể** chi phí năng lượng.
- **Tương thích** với việc chạy trên các thiết bị nhỏ gọn.

Nhóm nghiên cứu đã quyết định sử dụng kiến trúc **RWKV** thay vì Transformer tiêu chuẩn vì:

- **RWKV** là một kiến trúc lai (kết hợp **Transformer + RNN**).
- **Khả năng xử lý:** Vẫn giữ được khả năng mô hình hóa ngữ cảnh dài.
- **Hiệu suất tính toán:** Độ phức tạp tính toán chỉ là **$O(N)$ (tuyến tính)**, thay vì **$O(N^2)$ (bậc hai)** như cơ chế attention.
- **Tương thích với SNN:** RWKV **xử lý token tuần tự**, tương đồng với cách SNN hoạt động theo **timestep**, khiến nó **phù hợp hơn** để chuyển đổi sang SNN.

3. Phân tích Kiến trúc SpikeGPT

3.1 Leaky Integrate-and-Fire (LIF) Neuron

Đơn vị xử lý và phi tuyến hóa thông tin cơ sở của SpikeGPT là mô hình nơ-ron rò rỉ tích lũy và phát xung (Leaky Integrate-and-Fire - LIF). Hệ thống này thay thế hoàn toàn các hàm kích hoạt như ReLU trong kiến trúc mạng nhân tạo truyền thống, đảm nhận vai trò như một bộ chuyển đổi trạng thái linh hoạt: tiếp nhận dòng điện (định lượng số thực) và xuất ra tín hiệu kích thích (định lượng nhị phân).

Dựa trên các cơ sở toán học của SpikeGPT, tiến trình động học của nơ-ron LIF được mô tả thông qua hệ phương trình vi phân rời rạc (discrete-time) với ba giai đoạn cơ bản:

1. Cập nhật điện thế màng (Membrane Potential Update):

$$U[t] = H[t] + \beta \left(Y[t] - (H[t - 1] - U_{reset}) \right)$$

2. Cơ chế kích hoạt xung (Spike Firing):

$$S[t] = \Theta(U[t] - U_{threshold})$$

3. Thiết lập lại sau xung (Reset Mechanism):

$$H[t] = U[t] \cdot (1 - S[t])$$

Phân tích toán học:

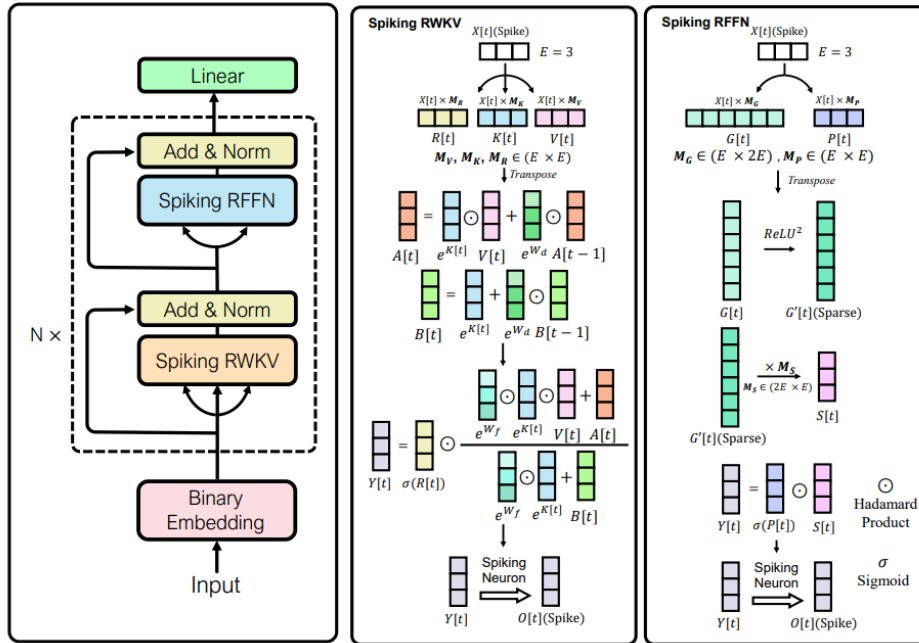
- $U[t]$ là điện thế màng tại thời điểm t , β là hệ số suy giảm (decay factor, thiết lập mặc định $\beta = 0.5$), và $Y[t]$ là tín hiệu đầu vào từ tầng trước (Spiking RWKV).
- $\Theta(\cdot)$ là hàm bước Heaviside (Heaviside step function). Nếu $U[t]$ vượt ngưỡng $U_{threshold} = 1$, nơ-ron sẽ phát ra xung nhị phân $S[t] = 1$.
- Sau khi phát xung ($S[t] = 1$), giá trị $H[t]$ (trạng thái ẩn) sẽ bị triệt tiêu về 0 (do $U_{reset} = 0$), đây gọi là cơ chế *hard reset* để tránh hiện tượng phát xung liên tục.

Giải quyết tính phi đạo hàm (Non-differentiability): Hàm $\Theta(\cdot)$ có đạo hàm bằng 0 tại mọi nơi, làm cho thuật toán backpropagation thất bại. SpikeGPT giải quyết bằng cách dùng **đạo hàm thay thế (Surrogate Gradient)** trong quá trình lan truyền ngược, cụ thể là đạo hàm của hàm arctangent:

$$\sigma'(x) = \frac{\alpha}{2(1 + (\frac{\pi}{2}\alpha x)^2)}$$

$$\sigma'(x) = \frac{1}{\pi \arctan(\pi x) + \frac{1}{2}}$$

hoặc dạng tương đương $\sigma'(x) = \frac{1}{\pi \arctan(\pi x) + \frac{1}{2}}$. Việc này cung cấp một công cụ ước lượng gradient chệch (biased gradient estimator) nhưng mượt mà, giúp mạng có thể cập nhật trọng số hiệu quả.



3.2 Model Architecture (Kiến trúc Tổng thể)

Nhằm vượt qua giới hạn độ phức tạp $O(N^2)$ của cơ chế Tự chú ý (Self-Attention) phổ chuẩn, SpikeGPT tích hợp một biến thể chú ý tuyến tính (linear attention) kế thừa từ kiến trúc **RWKV (Receptance Weighted Key Value)**. Mô hình kiến trúc này kết hợp năng lực mô hình hóa ngữ cảnh song song đặc thù của Transformer và khả năng suy luận tuyến tính đệ quy của Mạng nơ-ron Hồi quy (RNN).

Trong khuôn khổ phân hệ **Spiking Token Mixer**, giá trị chú ý tại bước thời gian t (ký hiệu là wkv_t) được ước lượng bằng định dạng số thực liên tục trước khi truyền vào nơ-ron LIF, tuân thủ hệ phương trình chuẩn hóa của RWKV:

Dòng chảy thông tin trong mạng đi qua các tầng được định nghĩa một cách tuyến tính và hồi quy. Cho một đầu vào $I \in \mathbb{R}^{T \times d}$, chuỗi biến đổi hình thức được định nghĩa như sau:

$$X_0 = \text{BE}(I)$$

$$X'_l = \text{SRWKV}(X_l - 1) + X_{l-1} \quad \text{với } l = 1 \dots L$$

$$X_l = \text{SRFFN}(X'_l) + X'_l \quad \text{với } l = 1 \dots L$$

$$Y = \text{GH}(X_L)$$

Phân tích toán học:

- $\text{BE}(\cdot)$ là tầng Nhúng nhị phân (Binary Embedding).
- Các khối bao gồm **SRWKV (Spiking Receptance Weighted Key Value)** đóng vai trò trộn thông tin (token mixer) và **SRFFN (Spiking Receptance Feed-Forward Network)** đóng vai trò trộn kênh đặc trưng (channel mixer).
- Đặc biệt, kết nối thặng dư (residual connections) sử dụng cấu hình **SEW-ResNet** (Spike-Element-Wise ResNet). Việc thiết kế cộng trực tiếp đầu vào X_{l-1} (vốn đã là các số nguyên/xung thừa thớt) với đầu ra của SRWKV đảm bảo rằng dữ liệu chảy qua các tầng mạng không bị biến đổi thành số thực (floating-point), duy trì hoàn toàn tính chất sparse integer spikes của SNNs.
- Khối SRWKV giải quyết độ phức tạp $O(T^2)$ của hàm Softmax truyền thống bằng cách sử dụng tích chập 1-D dọc theo chiều thời gian, phân rã ma trận Attention qua các vector suy giảm W theo hướng truy hồi (RNN form), biến phép nhân ma trận tĩnh thành các phép tính phần tử (element-wise products) tương thích cực tốt với phần cứng SNN.

3.3 Binary Embedding

Đề nhất quán với đặc tính kích hoạt nhị phân (binary activations) của mạng nơ-ron xung (SNNs), dữ liệu văn bản không thể được đưa vào dưới dạng các vector số thực liên tục như Transformer truyền thống. Thay vào đó, SpikeGPT đề xuất kỹ thuật **Nhúng nhị phân (Binary Embedding - BE)** nhằm chuyển đổi trực tiếp đầu ra của tầng nhúng thành chuỗi xung nhị phân ngay từ cửa ngõ mô hình.

Cơ chế này được xử lý qua hai pha toán học riêng biệt:

- **Pha lan truyền xuôi (Feed-Forward):** Mô hình áp dụng hàm bước nhảy Heaviside $\Theta(\cdot)$ để ánh xạ ma trận nhúng số thực $I \in \mathbb{R}^{T \times d}$ thành tensor xung $X_0 \in 0, 1^{T \times d}$. Công thức biến đổi hình thức được định nghĩa như sau: $X_0 = \text{BE}(I) = \Theta(I - \text{Threshold})$. Bước ép kiểu cứng (hard thresholding) này đóng vai trò then chốt, đảm bảo toàn bộ luồng tín hiệu (activations) đi vào các khối Spiking RWKV phía sau hoàn toàn là các bit 0, 1, cho phép SNN loại bỏ hoàn toàn phép nhân ma trận trữu nặng trong quá trình suy luận.
- **Pha lan truyền ngược (Backward Propagation):** Sự đánh đổi của hàm $\Theta(\cdot)$ là nó không thể vi phân (đạo hàm bằng không ở mọi nơi), làm cho thuật toán Backpropagation thất bại và không thể cập nhật trọng số. Để giải quyết điểm nghẽn giải tích này, tầng Binary Embedding lại tiếp tục áp dụng kỹ thuật **Đạo hàm thay thế (Surrogate Gradient)** tương tự như ở nơ-ron LIF. Cụ thể, đạo hàm của hàm arctangent

$$\sigma'(x) = \frac{1}{\pi \arctan(\pi x) + \frac{1}{2}}$$

được sử dụng làm bộ ước lượng gradient mượt:

Sự kết hợp giữa biến đổi Heaviside ở chiều xuôi và Đạo hàm thay thế ở chiều ngược là một thiết kế thỏa hiệp toán học xuất sắc. Nó ép buộc dữ liệu tuân thủ nghiêm ngặt bằng thông 1-bit để tận dụng hiệu năng của phần cứng thần kinh, nhưng vẫn duy trì được khả năng đạo hàm để cập nhật không gian vector từ vịnh (continuous weights) trong quá trình huấn luyện.

Nam

3.4 Xử lý hiệu quả các chuỗi có độ dài thay đổi bằng cách sử dụng Spiking RWKV

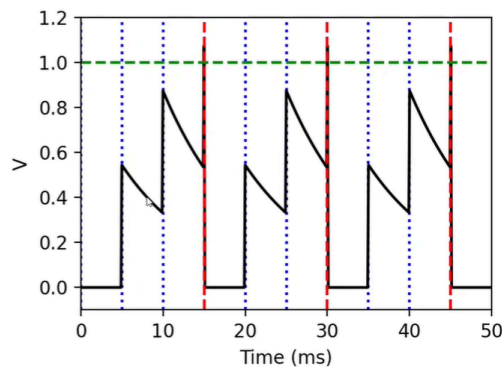
1. Những Hạn chế Cốt lõi của Cơ chế Tự chú ý (Self-Attention) đối với Mạng Nơ-ron Phát xung (SNN):

$$f(X) = \text{softmax} \left(\frac{Q(K)^T}{\sqrt{d_k}} \right) V, Q = XM_Q, K = XM_K, V = XM_V \quad (3)$$

Here, $M_Q \in \mathbb{R}^{d \times d_k}$, $M_K \in \mathbb{R}^{d \times d_k}$, and $M_V \in \mathbb{R}^{d \times d_v}$ represent linear transformations, while d_k and d_v signify the dimensions of the key and value vectors, respectively. This mechanism enables dynamic information mixing across token dimensions and accommodates sequences of variable lengths by leveraging dynamic matrix-matrix multiplication ($Q(K)^T$) and the softmax function (which necessitates access to the entire sequence).

- **Mâu thuẫn với Bản chất Hồi quy:**

- Self-Attention yêu cầu truy cập đồng thời vào *toàn bộ* chuỗi đầu vào để mô hình hóa ngôn ngữ.
- Điều này không phù hợp với SNN, vốn hoạt động **theo sự kiện** và chỉ có thể tạo ra **xung điện** (spikes) dựa trên thông tin từ các bước thời gian (*time-steps*) *trước* đó.



- **Chi phí Tính toán Cao:**

- Self-Attention dựa vào các phép nhân ma trận động (*dynamic matrix-matrix multiplication*) tốn kém.
- Việc áp dụng vào SNN sẽ tạo ra gánh nặng tính toán lớn, làm mất đi lợi thế giảm thiểu năng lượng của SNN.

- **Thách thức Mở rộng Mô hình:**

- Việc bổ sung chiều thời gian để nơ-ron SNN xử lý chuỗi dữ liệu theo cơ chế truyền tiến (*feed forward*) sẽ làm kích thước mô hình tăng tỷ lệ thuận -> Điều này gây khó khăn lớn khi muốn mở rộng mô hình lên quy mô hàng trăm tỷ tham số.

2. Giải pháp: Mô đun Spiking RWKV (SRWKV):

The subsequent step involves generating the output using the following equation:

$$Y[t+1] = \mathcal{SN} \left(\sigma(R[t]) \odot \frac{\exp(W_f) \odot \exp(K[t]) \odot (V[t] + A[t])}{\exp(W_f) \odot \exp(K[t]) + B[t]} \right) \quad (4)$$

The hidden states A and B are defined as:

$$A[t] = \exp(K[t]) \odot (V[t] + \exp(W_d) \odot A[t-1]) \quad (5)$$

and

$$B[t] = \exp(K[t-1]) + \exp(W_d) \odot B[t-1] \quad (6)$$

- **Xử lý Dữ liệu Tuần tự (Bước thời gian):**

- SRWKV chỉ nhận đầu vào $X[t]$ ứng với từng bước thời gian t , thay vì toàn bộ chuỗi X .
- Thiết kế này hoàn toàn tương thích với cấu trúc của mạng SNN.

- **Sử dụng Phép nhân Từng phần tử (*Element-wise Products*):**

- SRWKV vẫn tạo ra các ma trận R , K , V như Self-Attention nhưng đã loại bỏ hoàn toàn các phép nhân ma trận tốn kém.
- Thay vào đó, SRWKV sử dụng các phép toán nhân từng phần tử để trao đổi thông tin giữa các token.

- **Chiến lược Suy giảm Liên tục (*Continuous Decay Strategy*):**

- SRWKV không tự động điều chỉnh bản đồ chú ý theo đầu vào mà sử dụng các trạng thái ẩn $A[t]$ và $B[t]$ để lưu trữ thông tin quá khứ.
- Thông tin quá khứ được kiểm soát bằng hai vector học được: W_d (làm giảm dần ảnh hưởng của thông tin cũ) và W_f (đánh trọng số cho thông tin hiện tại).

- **Không cần Mặt nạ (*Masking*):**

- Trong quá trình sinh ngôn ngữ, Self-Attention cần che đi một nửa bản đồ chú ý để tránh rò rỉ thông tin từ tương lai.
- SRWKV và SNN, với bản chất hồi quy một chiều tự nhiên, giải quyết trực tiếp vấn đề này mà không cần sử dụng mặt nạ.

3.5 Mạng truyền thẳng tiếp nhận xung (Spiking Receptance Feed-Forward Networks - SRFFN)

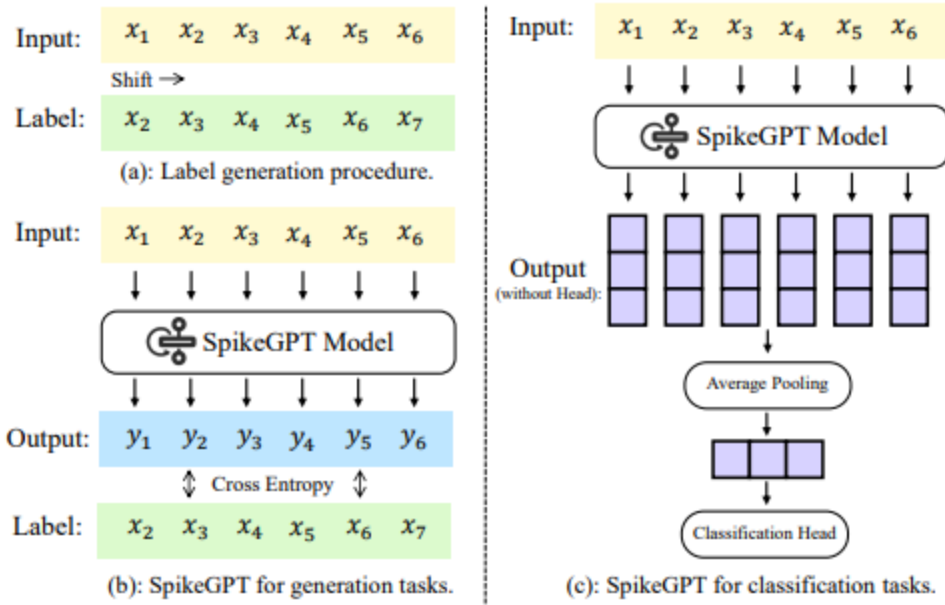


Figure 2: Training SpikeGPT for NLG and NLU tasks.

Vai trò và Nguyên tắc hoạt động:

- **Vị trí:** SRFFN là một mạng truyền thẳng kết nối đầy đủ (fully connected feed-forward network) được tích hợp trong mỗi khối của mô hình SpikeGPT.
- **Đầu vào:** Mô đun này tiếp nhận dữ liệu đã được chuẩn hóa và áp dụng kỹ thuật dịch chuyển token (token-shifted) từ khối Spiking RWKV liền trước.
- **Cơ chế Cổng (Gating Mechanism):** Về cốt lõi, SRFFN là một biến thể của kiến trúc

Cổng tuyến tính (Gated Linear Unit - GLU). Nó sử dụng hàm Sigmoid ($\sigma(M_P X[t])$) làm cơ chế cổng, cho phép kiểm soát chặt chẽ lượng thông tin được truyền vào mô hình.

Cấu trúc và Biến đổi Toán học:

$$Y'[t] = \mathcal{SN}(\sigma(M_P X[t]) \odot M_S(\text{Activation}(M_G X[t]))) \quad (7)$$

where $\mathcal{SN}$ represents spiking neuron layers, $Y'[t]$ denotes the output of SRFFN at time-step t which is then passed to the spiking neuron (Eq. 1).

- **Phép biến đổi:** Quá trình tính toán trong SRFFN bao gồm ba phép biến đổi tuyến tính sử dụng các ma trận trọng số có thể học được là M_P, M_G và M_S .
- **Kích thước Lớp Ẩn:** Để đảm bảo tính tương thích với tham số của GEGLU (một phiên bản GLU phổ biến trong kiến trúc Transformer), kích thước lớp ẩn H của SRFFN được đặt lớn gấp bốn lần kích thước đầu vào (4d).
- **Hàm Kích hoạt:** Tùy thuộc vào phiên bản cụ thể của SpikeGPT, SRFFN có thể sử dụng các hàm kích hoạt khác nhau, bao gồm ReLU2 (bình phương của hàm ReLU) hoặc các nơ-ron phát xung (Spiking Neurons - SN).

Duy trì Đặc tính Phát xung:

- **Bộ lọc LIF:** Sau khi dữ liệu đi qua quá trình truyền thẳng trong SRFFN, đầu ra sẽ được nạp tiếp vào một nơ-ron Leaky Integrate-and-Fire (LIF).
- **Mục tiêu:** Bước xử lý sau cùng này là yếu tố then chốt giúp mô hình bảo toàn các đặc điểm phát xung nhị phân ở cấp độ khối. Nhờ đó, nó duy trì được độ thưa thớt (sparsity) – tính chất thiết yếu giúp mạng nơ-ron phát xung (SNN) đạt được hiệu quả năng lượng vượt trội.

3.6 Phân tích Tiêu thụ Năng lượng Lý thuyết

Đánh giá khối Spiking RWKV:

Khối Spiking RWKV được cấu thành từ hai yếu tố chính: lớp Spiking MLP (tiếp nhận đầu vào là các xung thưa thớt) và các phép nhân từng phần tử với độ chính xác đầy đủ.

- **Ưu điểm của đầu vào dạng xung:** Nhờ có đầu vào xung, các lớp MLP có khả năng chuyển đổi phép nhân ma trận (vốn tiêu tốn nhiều năng lượng) thành các phép toán cộng thưa thớt.
- **Hiệu quả trên phần cứng Neuromorphic:** Hoạt động nhân từng phần tử không ở dạng nhị phân chỉ chiếm một tỷ lệ nhỏ. Tất cả các hoạt động này đều được các chip neuromorphic hỗ trợ một cách hiệu quả.

Đánh giá khối SRFFN:

Khối SRFFN chủ yếu bao gồm các lớp MLP và các phép nhân từng phần tử.

- **Độ thưa thớt động:** Việc sử dụng hàm kích hoạt ReLU thay vì nơ-ron phát xung (LIF) vẫn giúp khối này tạo ra độ thưa thớt động.
- **Tương thích hoàn toàn với phần cứng Neuromorphic:** Các kích hoạt có thể được lượng tử hóa thành các số nguyên (ví dụ: 8-bit). Do các chip neuromorphic hiện đại có thể xử lý các xung phân cấp lên đến 32-bit số nguyên, khối SRFFN hoàn toàn tương thích vì nó chỉ yêu cầu các phép toán cộng để thực thi.

So sánh hiệu năng tiêu thụ tổng thể:

Dựa trên phân tích năng lượng lý thuyết (Bảng 1), sự kết hợp của các thành phần trên cho phép mô hình SpikeGPT giảm thiểu tổng năng lượng tiêu thụ tới **32,2 lần** so với mô hình Vanilla GPT truyền thống.

- **Nguyên nhân giảm thiểu năng lượng:** Sự sụt giảm đáng kể này là nhờ tất cả các đầu vào của các module (trừ thao tác nhân từng phần tử) đều ở dạng xung, giúp cắt giảm một lượng lớn các phép toán dấu phẩy động (FLOPs).

reflected in the vanilla GPT-to-SpikeGPT (V/S) ratio.

		Vanilla GPT (Radford et al., 2018) (with GLU (Dauphin et al., 2017))	SpikeGPT (This work)	Energy Consumption (pJ)		Ratio
				Vanilla GPT	SpikeGPT	V/S
Attention	$Q/R, K, V$	$E_{MAC} \cdot 3Td^2$	$E_{AC} \cdot \hat{R} \cdot 3Td^2$	1.09×10^{10}	3.25×10^8	33.3×
	$f(Q/R, K, V)$	$E_{MAC} \cdot 2T^2d$	$E_{MAC} \cdot 7Td$	8.50×10^7	4.95×10^7	1.71×
	Scale	$E_{MAC} \cdot T^2$	-	4.25×10^7	-	-
	Softmax	$E_{MAC} \cdot 2T^2$	-	8.50×10^7	-	-
FFN	Layer 1	$E_{MAC} \cdot FL_{MLP1}$	$E_{AC} \cdot \hat{R} \cdot FL_{MLP1}$	3.62×10^9	1.09×10^8	33.3×
	Layer 2	$E_{MAC} \cdot FL_{MLP2}$	$E_{AC} \cdot \hat{R} \cdot FL_{MLP2}$	1.45×10^{10}	4.35×10^8	33.3×
	Layer 3	$E_{MAC} \cdot FL_{MLP3}$	$E_{AC} \cdot \hat{R} \cdot FL_{MLP3}$	3.62×10^9	1.09×10^8	33.3×
Overall	-	-	-	3.29×10^{10}	1.02×10^9	32.2×

Đồ án

Báo Cáo Triển Khai Hệ Thống Chatbot Agentic Trong Quản Lý Thương Mại

1. Giới thiệu

Trong bối cảnh bán lẻ và thương mại điện tử hiện đại, các nhà quản lý cửa hàng thường xuyên phải xử lý một lượng lớn dữ liệu động: từ thông tin sản phẩm, tình trạng tồn kho, đến tiến độ đơn hàng và báo cáo doanh thu. Quy trình truyền thống đòi hỏi người quản lý phải thao tác trực tiếp với hệ thống quản trị, hoặc sử dụng các truy vấn cơ sở dữ liệu phức tạp. Điều này không chỉ gây tiêu tốn thời gian mà còn tiềm ẩn nhiều rủi ro sai sót trong quá trình trích xuất số liệu.

Với sự phát triển vượt bậc của các Mô hình Ngôn ngữ Lớn (LLMs) và kiến trúc AI Agentic, việc xây dựng các trợ lý ảo có khả năng tương tác trực tiếp với hệ thống cơ sở dữ liệu thông qua các công cụ (Tool Calling) đã trở nên khả thi. Thay vì điều hướng qua nhiều giao diện hoặc viết mã SQL/GraphQL, người dùng chỉ cần đưa ra yêu cầu bằng ngôn ngữ tự nhiên. Hệ thống sẽ tự động phân tích ngữ nghĩa, gọi các API tương ứng để trích xuất dữ liệu, và tổng hợp thành câu trả lời trực quan, chính xác.

2. Giải pháp: Hệ thống Chatbot Agentic

Dự án tập trung nghiên cứu và phát triển một hệ thống Chatbot Agentic chuyên biệt phục vụ tác vụ quản lý thương mại. Hệ thống đóng vai trò như một tác tử trung gian thông minh với các năng lực cốt lõi:

- **Truy xuất dữ liệu tự động:** Tương tác trực tiếp với cơ sở dữ liệu và hệ thống quản lý (như danh mục sản phẩm, đơn hàng) thông qua các Tool APIs.
- **Định tuyến công cụ (Tool Routing):** Tự động suy luận và lựa chọn công cụ (tool) phù hợp nhất dựa trên ý định trong câu hỏi của người dùng.
- **Tổng hợp và Trình bày:** Xử lý dữ liệu thô trả về từ API, tóm tắt và trình bày lại cho người quản lý dưới định dạng ngôn ngữ tự nhiên, dễ đọc, dễ hiểu.

3. Mục tiêu Dự án

Việc triển khai hệ thống hướng đến ba mục tiêu trọng tâm:

- **Tối ưu hóa thời gian:** Giảm thiểu đáng kể thời gian thao tác thủ công của người quản lý cửa hàng trên các phần mềm quản trị.
- **Đơn giản hóa quy trình:** Xóa bỏ rào cản kỹ thuật trong việc truy xuất dữ liệu, cho phép người dùng không có chuyên môn IT vẫn có thể tra cứu số liệu phức tạp.
- **Nâng cao năng lực ra quyết định:** Cung cấp thông tin theo thời gian thực một cách chính xác, hỗ trợ các quyết định kinh doanh nhanh chóng và hiệu quả.

4. Giới thiệu các Mô hình Ngôn ngữ áp dụng

Để thực thi tác vụ Agentic và Tool Calling, dự án tiến hành thử nghiệm và so sánh ba kiến trúc mô hình khác nhau, mỗi mô hình mang những đặc điểm kỹ thuật riêng biệt:

4.1. SpikeGPT

SpikeGPT là một kiến trúc tiên tiến kết hợp giữa Mạng nơ-ron xung (Spiking Neural Networks - SNNs) và cơ chế ngôn ngữ hiện đại, hoạt động dựa trên các sự kiện xung nhịp để xử lý luồng dữ liệu.

- **Ưu điểm:**
 - **Hiệu suất năng lượng cực cao:** Do tính chất thưa thớt của SNN, mô hình tiết kiệm đáng kể tài nguyên tính toán và chi phí duy trì hệ thống so với các mô hình Transformer truyền thống.
 - **Độ trễ thấp:** Phù hợp để triển khai các hệ thống yêu cầu phản hồi nhanh (real-time) hoặc chạy trên các thiết bị/môi trường có giới hạn về phần cứng.
 - **Xử lý luồng (Streaming):** Kiến trúc có khả năng nạp và xử lý token theo dạng chuỗi thời gian một cách tự nhiên.
- **Nhược điểm:**
 - **Độ phức tạp trong huấn luyện:** Cần phương pháp mã hóa dữ liệu đặc thù (như biến đổi dữ liệu sang dạng binary/spike) và việc hội tụ gradient khó khăn hơn.
 - **Hệ sinh thái:** Các thư viện và tài liệu hỗ trợ chưa phong phú bằng hệ sinh thái của kiến trúc Transformer.

4.2. GPT-2 Small (124M Parameters)

GPT-2 Small là phiên bản nhẹ nhất trong họ mô hình sinh văn bản của OpenAI, sử dụng kiến trúc Decoder-only Transformer.

- **Ưu điểm:**
 - **Triển khai nhanh chóng:** Kích thước siêu nhỏ giúp mô hình dễ dàng được tải và tinh chỉnh (fine-tune) trên các máy tính cá nhân hoặc hệ thống đám mây tiêu chuẩn mà không cần GPU đắt tiền.
 - **Làm Baseline lý tưởng:** Rất phù hợp để thiết lập mức cơ sở (baseline) khi đánh giá các tác vụ NLP cơ bản trước khi chuyển sang mô hình phức tạp hơn.
- **Nhược điểm:**
 - **Năng lực suy luận hạn chế:** Khó có thể xử lý các logic Tool Calling phức tạp, đa bước hoặc các câu hỏi chứa nhiều tham số ẩn.
 - **Dễ bị ảo giác (Hallucination):** Thường xuyên sinh ra các tham số (arguments) không chính xác hoặc gọi sai tên công cụ do dung lượng tham số thấp.

4.3. Fine-tuned GPT-2 Medium (355M Parameters)

Bản nâng cấp GPT-2 Medium với 355 triệu tham số, được nhóm dự án tiến hành tinh chỉnh (fine-tune) sâu trên bộ dữ liệu Tool Calling đặc thù của ngành thương mại.

- **Ưu điểm:**
 - **Cân bằng giữa hiệu năng và tài nguyên:** Khả năng nắm bắt cú pháp JSON và lập luận (reasoning) để chọn công cụ tốt hơn hẳn bản Small, nhưng vẫn đủ nhẹ để vận hành mượt mà với chi phí thấp.
 - **Độ chính xác cao trong miền hẹp:** Nhờ quá trình fine-tune với dữ liệu hội thoại đa lượt, mô hình có khả năng trích xuất các đối số (arguments) từ truy vấn người dùng chính xác hơn nhiều so với việc dùng zero-shot.
- **Nhược điểm:**
 - **Giới hạn về Context Window:** Kiến trúc cũ có độ dài ngữ cảnh hạn chế (thường là 1024 token), gây khó khăn khi phải chứa nhiều thông tin từ lịch sử hội thoại hoặc kết quả mô tả API dài.
 - **Khả năng tổng quát hóa:** Chịu giới hạn về mặt kiến trúc so với các LLM hiện đại, nên hiệu suất có thể giảm sút nhanh chóng khi đối mặt với các cấu trúc lệnh (prompt) nằm ngoài phân phối dữ liệu huấn luyện.

5. Bộ dữ liệu Tool Calling

Để huấn luyện và tinh chỉnh các mô hình trên (đặc biệt là thiết lập luồng Agent), dự án đã tự xây dựng một bộ dữ liệu hội thoại đa lượt chất lượng cao, mô phỏng sát nhất các tác vụ quản lý nghiệp vụ thực tế.

5.1. Thu thập Dữ liệu & Định dạng

Bộ dữ liệu bao quát các truy vấn phổ biến trong nghiệp vụ quản lý: kiểm tra tồn kho, tra cứu trạng thái đơn hàng, đối soát doanh thu và thông tin chi tiết sản phẩm. Mỗi mẫu dữ liệu mô phỏng một quy trình hội thoại trọn vẹn giữa hệ thống và người quản lý.

Cấu trúc chuẩn của một mẫu được tổ chức dưới dạng JSON minh bạch về vai trò (role):

- **user:** Truy vấn hoặc lệnh từ người quản lý.
- **assistant:** Phản hồi của Agent (thường chứa chỉ thị gọi API).
- **tool_calls:** Mảng chứa tên công cụ (name) và các đối số tương ứng (arguments) dạng JSON.
- **tool:** Dữ liệu thô trả về từ hệ thống sau khi chạy API.

Ví dụ cấu trúc:

```
{
  "id": "sample_1801",
  "description": "Kiểm tra tồn kho cho mã sản phẩm cụ thể",
  "tool_used": ["check_inventory"],
  "messages": [
    { "role": "user", "content": "Kho còn bao nhiêu sản phẩm P100?" },
    {
      "role": "assistant",
```

```

    "content": null,
    "tool_calls": [
      {
        "name": "check_inventory",
        "arguments": {"product_id": "P100"}
      }
    ]
  }
]
}

```

5.2. Quy trình Xây dựng Dữ liệu

1. **Tạo tập dữ liệu vàng (Gold Standard):** Các chuyên gia tự tay xây dựng một tập hợp các mẫu hội thoại hoàn hảo để làm chuẩn mực.
2. **Sinh dữ liệu tự động (Synthetic Generation):** Sử dụng các LLM mạnh (như Gemini Pro) làm đầu vào để mô phỏng và mở rộng quy mô bộ dữ liệu dựa trên tập "vàng", tạo ra hàng ngàn mẫu JSON đa dạng.
3. **Đánh giá chéo (IAA):** Dữ liệu sinh ra được các thành viên trong nhóm rà soát chéo. Áp dụng phương pháp Đồng thuận giữa các người gán nhãn (Inter-Annotator Agreement - IAA) để lọc bỏ các phản hồi sai lệch, ảo giác.
4. **Lưu trữ:** Đóng gói toàn bộ tập dữ liệu đã xác thực thành các tệp JSON có cấu trúc để quản lý version và nạp vào pipeline huấn luyện.

5.3. Kiểm chứng và Đảm bảo Chất lượng

Trước khi đưa vào huấn luyện, toàn bộ file JSON phải vượt qua quy trình kiểm thử tự động nghiêm ngặt:

- **Tính toàn vẹn:** Đảm bảo không có trường dữ liệu nào bị khuyết thiếu (`description`, `tool_used`, `messages`...).
- **Tính nhất quán của Tool:** Đối chiếu `tool_used` ở cấp độ metadata phải khớp hoàn toàn với các hàm được gọi bên trong mảng `tool_calls` của Agent.
- **Độ chính xác của Argument:** Kiểm tra chéo xem các tham số (ví dụ: `product_id`) Agent trích xuất có hoàn toàn trùng khớp với thực thể được người dùng nhắc đến trong câu hỏi gốc hay không.

5.4. Tiền xử lý dữ liệu

- **Lọc độ dài:** Loại bỏ các mẫu hội thoại có độ dài vượt quá giới hạn (ví dụ: `token_length > 600`) để phù hợp với Context Window của các mô hình nhỏ (như GPT-2).
- **Tokenization:** Mã hóa văn bản thành các token bằng tokenizer chuyên dụng của từng mô hình tương ứng.
- **Data Augmentation:** Tăng cường dữ liệu bằng các kỹ thuật xáo trộn thứ tự câu hỏi, thay thế từ đồng nghĩa hoặc đổi tham số giả để chống overfitting và tăng tính đa dạng.

- **Chuyển đổi đặc thù cho SpikeGPT:** Tiến hành xử lý thêm một bước biến đổi chuỗi dữ liệu sang dạng nhị phân (binary encoding) để tương thích với cơ chế xử lý theo xung (spike) của mạng SNN.

6. Training & Evaluation

Model	Valid JSON (%)	Intent Accuracy (%)	Args Exact Match (%)	Precision	Recall	F1 Score
SPIKEGPT						
SpikeGPT NoHeadQK (epoch 78)	100.00	86.62	0.00	0.8781	0.8662	0.8659
SpikeGPT Scratch (epoch 78)	95.98	84.64	1.78	0.8944	0.8464	0.8655
SpikeGPT Scratch (epoch 220)	91.52	81.24	4.68	0.9022	0.8124	0.8463
SpikeGPT Finetune (epoch 220)	99.80	94.10	13.94	0.9459	0.9410	0.9415
BASELINE						
GPT2_Small	99.98	96.52	68.30	0.9671	0.9652	0.9648
GPT2_Medium	100.00	96.78	67.76	0.9696	0.9678	0.9678

7. Demo

Tab 6

Dưới đây là 3 lý do kỹ thuật chính giải thích cho sự chênh lệch này:

1. Cơ chế Self-Attention (Tự chú ý) so với Trạng thái Hồi quy (Recurrent State)

- **GPT-2 (Transformer):** Sử dụng cơ chế Multi-Head Self-Attention. Cơ chế này cho phép mô hình nhìn vào toàn bộ chuỗi văn bản đầu vào cùng một lúc. Khi cần trích xuất một tham số (ví dụ: tên một biến hoặc một ID cụ thể), mô hình có thể tính toán trọng số chú ý trực tiếp và chính xác đến đúng từ đó trong ngữ cảnh, bất kể nó nằm ở đâu. Không có thông tin nào bị "quên".
- **SpikeGPT (RWKV/SNN):** Hoạt động giống như một mạng nơ-ron hồi quy (RNN). Nó đọc thông tin tuần tự và nén toàn bộ lịch sử ngữ cảnh vào một "trạng thái ẩn" (hidden state). Do đặc tính "leaky" (rò rỉ điện thế) của các nơ-ron xung, các chi tiết nhỏ lẻ nhưng quan trọng (như một dãy số, một chuỗi ký tự chính xác cần đưa vào JSON) rất dễ bị mờ nhạt hoặc biến mất khi truyền qua các bước thời gian.

2. Độ phân giải dữ liệu: Liên tục (Continuous) vs. Nhị phân (Binary)

- **GPT-2:** Xử lý và truyền tải thông tin bằng các giá trị số thực dấu phẩy động (FP16 hoặc FP32). Sự thay đổi nhỏ nhất trong ngữ nghĩa đều có thể được biểu diễn chính xác bằng các con số thập phân, giúp mô hình giữ được độ sắc nét (precision) cực cao khi sinh ra các token chính xác cho tham số.
- **SpikeGPT:** Nén thông tin và giao tiếp giữa các lớp bằng các xung nhị phân (**chỉ có 0 và 1**). Quá trình lượng tử hóa khắc nghiệt này làm mất đi các sắc thái biểu diễn tinh tế. Nó đủ tốt để giữ lại "ý chính" (Intent) của câu, nhưng lại quá thô (coarse) để tái tạo chính xác từng ký tự (Arguments) mà một API hoặc Tool yêu cầu.

3. Sự chính xác trong quá trình Tối ưu hóa (Huấn luyện)

- **GPT-2:** Quá trình lan truyền ngược (Backpropagation) trong Transformer sử dụng đạo hàm toán học chính xác. Mô hình học được chính xác cách điều chỉnh trọng số để ánh xạ từ từ vựng đầu vào sang cấu trúc JSON đầu ra.
- **SpikeGPT:** Hàm tạo xung (Spike) là một hàm bước (Step function) không thể tính đạo hàm. Do đó, SpikeGPT phải dùng **Surrogate Gradient (Đạo hàm thay thế/xấp xỉ)** để huấn luyện. Sự "xấp xỉ" này tạo ra nhiễu (noise) trong quá trình cập nhật trọng số. Khi fine-tune cho một tác vụ đòi hỏi sự khớp chính xác 100% như Args Exact Match, lượng nhiễu này khiến mô hình rất khó hội tụ về mức độ chính xác tuyệt đối.

Tóm lại, **GPT2_Small** "thắng" vì nó giữ lại toàn bộ độ phân giải của dữ liệu và có khả năng truy cập trực tiếp vào bất kỳ vị trí nào trong câu. Trong khi đó, **SpikeGPT** đánh đổi độ chính xác của các chi tiết nhỏ gọn để lấy sự tối ưu về năng lượng tính toán.

Để chứng minh bằng toán học lý do vì sao SpikeGPT (kiến trúc SNN/RWKV) lại thua kém GPT-2 (kiến trúc Transformer) trong việc trích xuất chính xác tham số (Args Exact Match), chúng ta cần đặt các phương trình cốt lõi của hai mô hình này lên bàn cân.

Dưới đây là 3 góc độ toán học lý giải hiện tượng "rơi rụng thông tin chi tiết" của SpikeGPT.

1. Phương trình Trí nhớ: Tự chú ý (Self-Attention) vs. Phân rã theo thời gian (Time Decay)

Việc trích xuất đúng một tham số (ví dụ: lấy ra đúng ID `user_123` nằm ở đầu câu) phụ thuộc vào khả năng truy xuất lại vị trí đó.

Đối với GPT-2 (Transformer):

Cơ chế Self-Attention tính toán sự chú ý từ token đầu ra hiện tại (vị trí i) tới mọi token trong quá khứ (vị trí j):

$$\text{Output}_i = \sum_{j=1}^i \text{softmax}\left(\frac{q_i k_j^T}{\sqrt{d}}\right) v_j$$

- Ý nghĩa:** Ma trận softmax đảm bảo rằng nếu token j (chứa tham số quan trọng) có độ tương đồng cao với câu truy vấn q_i , trọng số chú ý $a_{i,j} \approx 1$. Phương trình này là **không suy giảm theo khoảng cách**. Dù tham số nằm cách đó 1000 tokens, GPT-2 vẫn có thể bốc chính xác v_j ra mà không bị nhiễu.

Đối với SpikeGPT (Kế thừa RWKV):

Kiến trúc này dùng một trạng thái ẩn (hidden state) h_t để nén quá khứ, dựa trên phương trình hồi quy tuyến tính:

$$h_t = e^{-w} \odot h_{t-1} + k_t \odot v_t$$

- Ý nghĩa:** Thông tin từ thời điểm $t-N$ (quá khứ) khi truyền đến hiện tại t sẽ bị nhân với hệ số suy giảm lũy thừa $(e^{-w})^N$.
- Hệ quả toán học:** Về mặt giải tích, $\lim_{N \rightarrow \infty} (e^{-w})^N = 0$ (vì vector $w > 0$). Điều này có nghĩa là thông tin chi tiết của các biến số (Arguments) sẽ **bị mờ dần theo cấp số nhân** khi ngữ cảnh dài ra. Mô hình nhớ được ý chính (Intent) nhờ các trạng thái tổng quát, nhưng đánh mất tính chính xác tuyệt đối của ký tự (Args).

2. Định lý Mất mát Lượng tử hóa (Quantization Loss)

Dữ liệu đầu ra của Agent gọi Tool (thường là JSON) đòi hỏi hàm lượng thông tin (Entropy) rất cao và độ nhiễu (Perplexity) cực thấp. Sai một dấu phẩy, toàn bộ chuỗi thất bại.

Đối với GPT-2:

Biểu diễn vector ở dạng số thực liên tục $x \in \mathbb{R}^d$ (ví dụ FP16). Không gian trạng thái là vô hạn, cho phép biểu diễn các ranh giới quyết định (decision boundaries) cực kỳ sắc nét.

Đối với SpikeGPT (SNN):

Mô hình sử dụng nơ-ron Leaky Integrate-and-Fire (LIF). Điện thế màng U_t được tích lũy:

$$U_t = \beta U_{t-1} + W X_t$$

Và đầu ra S_t bị ép qua hàm bước Heaviside Θ :

$$S_t = \Theta(U_t - \theta) = \begin{cases} 1 & \text{nếu } U_t \geq \theta \\ 0 & \text{nếu } U_t < \theta \end{cases}$$

- Hệ quả toán học:** Quá trình ánh xạ từ miền liên tục $U_t \in \mathbb{R}$ sang miền nhị phân $S_t \in \{0, 1\}$ sinh ra **Lỗi lượng tử hóa (Quantization Error)**:
- $E_q = \mathbb{E}[\|\text{Continuous} - S_t\|^2] > 0$
- Theo Định lý Shannon về dung lượng kênh truyền (Shannon Capacity), dung lượng thông tin tối đa (tính bằng bits) của một vector nhị phân thấp hơn rất nhiều so với vector

số thực. Mất mát E_q ở mỗi layer khi cộng dồn lại sẽ triệt tiêu các đặc trưng "tinh tế" nhất của dữ liệu, khiến mô hình không thể chốt hạ chính xác 100% các ký tự cấu thành nên một tham số cấu trúc.

3. Sự lệch chuẩn của Đạo hàm thay thế (Surrogate Gradient)

Để fine-tune mô hình học cách format JSON và trích xuất đúng biến, ta cần thuật toán Backpropagation (Lan truyền ngược) hoạt động hoàn hảo.

Sự bế tắc của SNN:

Đạo hàm của hàm Heaviside $\Theta(x)$ bằng 0 tại mọi nơi (trừ điểm 0 tiến tới $-\infty$).

$$\frac{\partial S_t}{\partial U_t} = 0 \quad (\forall U_t \neq \theta)$$

Nếu dùng đạo hàm này, gradient sẽ biến mất ngay lập tức (Vanishing Gradient), không thể cập nhật trọng số (Weight Update).

Giải pháp và Trái đấng của SpikeGPT:

SpikeGPT buộc phải xấp xỉ đạo hàm bằng một hàm liên tục (Surrogate Gradient), ví dụ như đạo hàm của hàm Sigmoid:

$$\frac{\partial S_t}{\partial U_t} \approx \sigma'(U_t - \theta)$$

- **Hệ quả toán học:** Sự xấp xỉ này tạo ra một độ lệch dư (Residual Bias) $\Delta = |\Theta' - \sigma'|$.
- Theo Quy tắc dây chuyền (Chain Rule) trong toán học, khi lan truyền ngược qua L lớp (Layers) và T bước thời gian (Time steps), độ lệch này bị nhân dồn lên:
- $\nabla W_{\text{error}} \propto \prod_{l=1}^L \prod_{t=1}^T \Delta_{l,t}$
- Sự tích tụ của ∇W_{error} tạo ra **Nhiều Gradient (Gradient Noise)**. Nhiễu này ngăn cản hàm mất mát (Loss Function) hội tụ về điểm cực tiểu toàn cục (Global Minimum) sắc nét - nơi yêu cầu mô hình phải khớp chính xác 100% cú pháp tham số. Thay vào đó, nó chỉ dừng lại ở một cực tiểu cục bộ (Local Minimum) tương đối - nơi mô hình chỉ đoán đúng "ý định" (Intent).

Kết luận

Toán học không nói dối. Việc **SpikeGPT** thất bại trong chỉ số **Args Exact Match** không phải do mô hình này "ngu" hơn GPT-2, mà là do sự hy sinh toán học có chủ đích: ép thông tin qua **hàm bước nhị phân** và **phân rã thời gian lũy thừa** để đổi lấy việc tiết kiệm 90% năng lượng tiêu thụ. Để gọi Tool chính xác, ở thời điểm hiện tại, toán học của Transformer (số thực + Self-Attention) vẫn là sự lựa chọn duy nhất.